

家計簿アプリ (Kakeibo)

個人の収支を記録・分析するための家計簿 Web アプリケーション。
AWS Solutions Architect Associate (SAA) のアウトプットとして、サーバーレス + 静的配信のクラウドネイティブ構成で構築。

概要

- **目的:** 月次レポート、日々の取引履歴、サブスクリプション管理ができる個人向け家計簿
- **位置づけ:** ポートフォリオ・SAA 学習成果物
- **特徴:** AWS で完結、月額 ¥3,800 程度の最適化構成

アーキテクチャ



構成サマリ

```
[フロントエンド]
ユーザー → Route 53 → CloudFront → S3 (Next.js 静的ファイル)
              + ACM (us-east-1)
              + OAC (S3 への直接アクセスを禁止)

[バックエンド]
ユーザー → Route 53 → API Gateway → Lambda (.NET 8 / ASP.NET Core)
              + ACM (ap-northeast-1)
              + CORS 制御
                  ↓ (VPC内、プライベートサブネット)
                  ↳ RDS PostgreSQL (5432)
                  ↳ VPC Endpoint (443)
                  ↓
                  Secrets Manager (DB認証情報)
```

技術スタック

フロントエンド

- Next.js 14 (CSR モード、`output: 'export'`)
- TypeScript
- React 18

バックエンド

- **.NET 8 + ASP.NET Core**
- **Entity Framework Core 8** (Npgsql)
- **Amazon.Lambda.AspNetCoreServer.Hosting** (Lambda 統合)

データベース

- **PostgreSQL 16** (Amazon RDS)

インフラ (AWS)

- **CloudFront + S3 + OAC** (静的配信)
- **API Gateway HTTP API + AWS Lambda** (サーバーレスバック)
- **RDS PostgreSQL + Secrets Manager + VPC Endpoint** (データ層)
- **Route 53 + ACM** (DNS + 証明書)

設計判断とその理由

ポートフォリオとしての「自分しか使わない」前提に対し、本番運用にも耐える設計を意図的に組み込んだ。

1. なぜ S3 + CloudFront を選んだか

候補	月額	採用判断
EC2 + Caddy + Next.js	\$38	× コスト高
Amplify Hosting	\$27	× マネージドすぎてブラックボックス
S3 + CloudFront	\$25	◎

選定理由:

- Next.js を CSR モードで実装することで、サーバー不要(月 \$10 削減)
- CloudFront のエッジ配信でグローバルに高速応答
- OAC により S3 への直接アクセスを禁止(セキュリティ)

2. なぜ Lambda を選んだか(EC2 でない理由)

- 「自分しか使わない」ためリクエスト課金が最適(常時稼働の EC2 より大幅に安い)
- Lambda の無料枠(月100万リクエスト)で実質コスト 0
- **SnapStart for .NET** によりコールドスタートを 5~10秒 → 1~2秒に短縮

3. なぜ RDS Proxy を使わないか

- Lambda の **Reserved Concurrency** を 10 に制限することで DB 接続枯渇を防止
- RDS Proxy(\$13/月)を省略してコスト最適化
- 個人利用では同時接続が問題にならない判断

4. なぜ NAT Gateway ではなく VPC Endpoint か

選択肢	月額
NAT Gateway	\$35
VPC Endpoint (Secrets Manager)	\$7

- Lambda が外部通信する必要があるのは Secrets Manager のみ
- VPC Endpoint で要件を満たしつつ月 \$28 削減
- プライベート通信を維持(AWS PrivateLink 経由)

5. なぜ Secrets Manager + 環境変数の使い分けか

機密性に応じた保管場所の使い分け:

情報	保管場所	理由
DB パスワード	Secrets Manager	IAM で守る必要あり
DB ホスト・名前	Lambda 環境変数	公開しても安全な接続情報
API URL	Next.js 環境変数 (<code>NEXT_PUBLIC_*</code>)	ブラウザに見えて当然

全部 Secrets Manager にすると起動が遅くなり、料金も増える(過剰設計)。

6. なぜ Single-AZ か

- 検証用・ポートフォリオ用途のため
- 本番では Multi-AZ 推奨だが、コスト 2倍になる(\$26)
- 「Single-AZ にしている理由を理解している」ことが SAA 的には大事

月額コスト

項目	月額 (USD)	円換算 (¥150/\$)
S3 (静的ファイル)	\$0.03	¥5
CloudFront	\$0 (無料枠)	¥0
Lambda	\$0 (無料枠)	¥0

項目	月額 (USD)	円換算 (¥150/\$)
API Gateway HTTP API	\$0 (無料枠)	¥0
RDS db.t4g.micro	\$13.14	¥1,970
RDS ストレージ (gp3 20GB)	\$2.76	¥410
VPC Endpoint (Secrets Manager)	\$7.30	¥1,100
Route 53 ホストゾーン	\$0.50	¥75
Secrets Manager	\$0.40	¥60
ACM 証明書 × 2	\$0	¥0
CloudWatch Logs	\$0.50	¥75
データ転送	\$0.50	¥75
合計	約 \$25	約 ¥3,800

別途: ドメイン代 年 \$13 (約 ¥160/月)

ディレクトリ構成

```
kakeibo/  
├── frontend/                # Next.js (CSR)  
│   ├── app/                # App Router  
│   ├── components/         # UI コンポーネント  
│   ├── lib/                # API クライアント  
│   ├── next.config.js      # output: 'export' 設定  
│   └── package.json  
│  
├── backend/                # ASP.NET Core  
│   ├── Controllers/        # API コントローラー  
│   ├── Services/           # ビジネスロジック  
│   ├── Models/             # データモデル  
│   ├── Program.cs          # エントリーポイント (Lambda 対応)  
│   └── Kakeibo.csproj  
│  
├── docs/  
│   └── architecture.png    # 構成図  
│  
└── README.md
```

API仕様(抜粋)

メソッド	パス	説明
GET	/api/Monthly	月次レポート一覧取得
POST	/api/Monthly	月次レポート作成 + トランザクション初期化 + サブスク登録
GET	/api/Transaction	取引履歴取得
POST	/api/Transaction	取引登録
PUT	/api/Transaction/{id}	取引更新
DELETE	/api/Transaction/{id}	取引削除

デプロイ手順

事前準備

```
bash

# AWS CLI セットアップ
aws configure

# .NET 8 SDK インストール
# https://dotnet.microsoft.com/

# dotnet lambda CLI ツール
dotnet tool install -g Amazon.Lambda.Tools

# Node.js 20+
node --version
```

1. ドメイン取得

AWS コンソール → Route 53 → 「ドメインの登録」 (年 \$13 程度)

2. インフラ構築(AWS コンソール手動)

以下の順序で構築:

1. VPC + プライベートサブネット × 2
2. セキュリティグループ × 3 (Lambda, RDS, VPC Endpoint)
3. Secrets Manager (DB 認証情報)
4. RDS PostgreSQL (db.t4g.micro)
5. VPC Endpoint (Secrets Manager)

6. IAM ロール (Lambda 用)
7. Lambda 関数 (ダミーコードで作成)
8. ACM 証明書 × 2 (CloudFront 用は us-east-1)
9. API Gateway HTTP API + カスタムドメイン
10. S3 バケット + バケットポリシー
11. CloudFront + OAC
12. Route 53 A レコード × 3

3. バックエンドデプロイ

```
bash

cd backend
dotnet lambda deploy-function kakeibo-api --region ap-northeast-1
```

4. フロントエンドデプロイ

```
bash

cd frontend
npm run build                # out/ ディレクトリ生成
aws s3 sync out/ s3://kakeibo-frontend/ --delete
aws cloudfront create-invalidation \
  --distribution-id <YOUR_DIST_ID> \
  --paths "/*"
```

5. DB マイグレーション(初回のみ)

```
bash

cd backend
# ローカルから RDS に一時接続してマイグレーション実行
dotnet ef database update --connection "Host=...;..."
```

SAA 試験で学んだトピック(本構成で実装)

ネットワーク

- ☑ VPC、サブネット(プライベート)、ルートテーブル
- ☑ セキュリティグループ(SG 同士の参照)
- ☑ VPC Endpoint (Interface 型) / AWS PrivateLink
- ☑ マルチ AZ 設計(サブネット配置)

コンピュータ

- ☑ AWS Lambda (VPC 内配置、Reserved Concurrency)
- ☑ Lambda SnapStart (コールドスタート対策)
- ☑ API Gateway HTTP API (REST API との違い)

データベース

- ☑ RDS PostgreSQL (db.t4g.micro、Single-AZ)
- ☑ Secrets Manager (機密情報管理、IAM 制御)

静的コンテンツ配信

- ☑ S3 (ストレージクラス、バケットポリシー)
- ☑ CloudFront (エッジ配信、TTL、ビヘイビア)
- ☑ Origin Access Control (OAC)
- ☑ マルチリージョン ACM (CloudFront 用は us-east-1)

セキュリティ

- ☑ IAM ロール (Lambda 用、最小権限)
- ☑ 機密情報と非機密情報の保管場所の使い分け
- ☑ プライベートサブネットによるネットワーク分離

DNS / 証明書

- ☑ Route 53 (ホストゾーン、A レコード、Alias レコード)
- ☑ ACM (DNS 検証、自動更新)

改善検討事項(本番運用時)

ポートフォリオ用に妥協している部分。本番運用するなら以下を検討:

項目	現状	本番推奨
RDS 冗長化	Single-AZ	Multi-AZ (+\$13/月)
RDS バックアップ	7日	14～30日
RDS 削除保護	無効	有効
削除時スナップショット	スキップ	取得必須
WAF	なし	CloudFront 前段に AWS WAF
Lambda 監視	基本ログのみ	X-Ray + CloudWatch Alarms
RDS Proxy	なし	アクセス増加時は追加
マルチリージョン DR	なし	必要に応じて検討

ライセンス

(任意で記載)

作者

(任意で記載)